

CourtDetection Module

1. Overview

The CourtDetection module is responsible for detecting and localizing the court keypoints in video frames. It's a fine-tuned ResNet-50 model to regress 14 court keypoints (x, y), which are later refined for higher precision. The predicted keypoints serve as a geometric reference for other modules such as PlayerTracking and BallTracking, enabling filtering of detections and contextual understanding of the court.

This module ensures consistent and accurate localization of the court, providing a geometric foundation for advanced analytics. We need to refine keypoints using classical computer vision

2. Dependencies

- **PyTorch**: for model definition and inference.
- **Torchvision**: for ResNet-50 backbone and preprocessing transforms.
- **OpenCV**: for visualization and keypoint refinement (sub-pixel accuracy).
- **NumPy**: for postprocessing and array manipulation.

3. Class: CourtDetection

Initialization

CourtDetection(model_path)

- **Parameters:**
 1. `model_path`: Path to the trained ResNet-50 weights for tennis court keypoint detection.

Note: The pretrained weights must be downloaded from the following link:
Trained tennis court keypoint model: [keypoints_model.pth - Google Drive](#)

Methods

Prediction Method

predict(image)

- **Purpose:** Predict 14 court keypoints from a single input frame.
- **Parameters:**
 1. `image`: Input frame in BGR format.
- **Output:** Array of 28 values (14 x, y pairs).

Visualization Methods

`draw_keypoints(image, keypoints)`

- **Purpose:** Draw predicted court keypoints on a single frame.
- **Parameters:**
 1. `image`: Frame to annotate.
 2. `keypoints`: List or array of court keypoints.
- **Output:** Annotated frame with keypoints and IDs.

`draw_keypoints_on_video(video_frames, keypoints)`

- **Purpose:** Draw predicted court keypoints across multiple video frames.
- **Parameters:**
 1. `video_frames`: List of frames.
 2. `keypoints`: Court keypoints to overlay.
- **Output:** Annotated video frames with overlaid keypoints.

4. Postprocessing: Keypoint Refinement

To enhance the accuracy of predicted keypoints, a refinement step is included using sub-pixel precision techniques. This reduces noise and improves alignment with court lines. You can find this process in this notebook: [Images/Keypoints/postProcessing.ipynb at main · Malk97sc/Images](#)

Function: `refine_keypoints_subpix(source, keypoints, win_size=10, zero_zone=(-1,-1), blur_ksize=3)`

- **Purpose:** Refine coarse keypoints predicted by the ResNet model.
- **Parameters:**
 1. `source`: Input frame (BGR format).
 2. `keypoints`: Raw predicted keypoints (list/array).
 3. `win_size`: Size of search window around each keypoint.
 4. `zero_zone`: Half of the size of the dead region in the middle of the search zone (default: (-1,-1)).
 5. `blur_ksize`: Gaussian blur kernel size before refinement.
- **Output:**
 1. `refined`: Refined keypoints with sub-pixel precision.
 2. `ok_mask`: Boolean mask indicating whether refinement succeeded for each point.

5. Notes

- The module is based on ResNet-50, modified for regression of 28 outputs (14 x-y pairs).
- Trained weights are required to achieve accurate predictions.
- Keypoint refinement is strongly recommended before integrating with PlayerTracking or BallTracking.
- The module assumes a standard tennis court with 14 labeled keypoints for consistency.
- This component provides the geometric foundation for the system's tracking and analytics pipeline.